

# PROYECTO FINAL – GRUPO 2

## API DE SEGUIMIENTO DE GASTOS PERSONALES

Junio 2026



**Universidad**  
Internacional  
de Valencia

### Titulación:

Máster en formación  
permanente en IA para el  
sector financiero

Curso académico

2025 – 2026

### Integrantes

Johan Javier Trujillo Gutiérrez

Gina Julieth Sanabria  
Maldonado

Paulo Ernesto Rodríguez

Cindy Marilú Pablo Espinoza

Felipe Pardo Parra

**Convocatoria:**  
Convocatoria

Primera

De:

Planeta Formación y Universidades

## DATOS IDENTIFICATIVOS DEL PROYECTO GRUPAL

| Nombre y apellidos              | Participación                                          |
|---------------------------------|--------------------------------------------------------|
| Johan Javier Trujillo Gutiérrez | Desarrollo e implementación del sistema                |
| Felipe Pardo Parra              | Desarrollo de funcionalidades y pruebas                |
| Gina Julieth Sanabria Maldonado | Integración y validación del proyecto                  |
| Paulo Ernesto Rodríguez         | Configuración y soporte técnico                        |
| Cindy Marilú Pablo Espinoza     | Documentación técnica, evidencias y presentación final |

## DISTRIBUCIÓN DE EPÍGRAFES

| Epígrafes                                   | Responsable(s) | Participación                                       |
|---------------------------------------------|----------------|-----------------------------------------------------|
| Introducción, contextualización y objetivos | Todo el grupo  | Redacción y revisión del planteamiento del proyecto |
| Requerimientos y arquitectura del sistema   | Todo el grupo  | Definición de la estructura general                 |
| Tecnologías, módulos y endpoints            | Todo el grupo  | Documentación técnica                               |
| Integración con Zapier y GitHub             | Todo el grupo  | Implementación y revisión                           |
| Evidencias, resultados y conclusiones       | Todo el grupo  | Organización y elaboración del informe final        |

## MECANISMOS DE COORDINACIÓN DEL GRUPO

El equipo de trabajo estableció una metodología colaborativa basada en la distribución de responsabilidades y la revisión conjunta de los avances del proyecto.

Para la coordinación se utilizaron las siguientes herramientas:

- **GitHub**, como repositorio principal para el control de versiones y almacenamiento del código fuente.
- **Google Drive**, para compartir documentación y realizar revisiones conjuntas.
- **Comunicación entre integrantes**, para coordinar las actividades y consolidar las diferentes contribuciones realizadas durante el desarrollo del proyecto.

Asimismo, cada integrante participó en la revisión y validación de las diferentes secciones del proyecto, permitiendo mantener una estructura organizada y coherente para la entrega final.

## ÍNDICE

|                                                   |   |
|---------------------------------------------------|---|
| 1. INTRODUCCIÓN.....                              | 1 |
| 2. CONTEXTUALIZACIÓN DEL PROBLEMA .....           | 2 |
| 3. OBJETIVO GENERAL.....                          | 3 |
| 4. OBJETIVOS ESPECÍFICOS .....                    | 3 |
| 5. REQUERIMIENTOS DEL SISTEMA.....                | 4 |
| 5.1 Requerimientos funcionales .....              | 4 |
| RF01. Registro de gastos .....                    | 4 |
| RF02. Consulta de gastos .....                    | 4 |
| RF03. Clasificación por categorías.....           | 4 |
| RF04. Generación de resúmenes .....               | 4 |
| RF05. Gestión de reglas de alerta .....           | 4 |
| RF06. Integración con herramientas externas ..... | 4 |
| RF07. Comunicación mediante Webhooks.....         | 4 |
| 5.2 Requerimientos no funcionales .....           | 4 |
| RNF01. Escalabilidad .....                        | 4 |
| RNF02. Mantenibilidad .....                       | 5 |
| RNF03. Rendimiento.....                           | 5 |
| RNF04. Portabilidad.....                          | 5 |
| RNF05. Colaboración .....                         | 5 |
| 6. ARQUITECTURA GENERAL DEL SISTEMA .....         | 6 |
| Usuario .....                                     | 6 |
| FastAPI.....                                      | 6 |
| Base de datos.....                                | 6 |
| Zapier.....                                       | 6 |
| Azure App Service .....                           | 6 |
| Flujo general del sistema.....                    | 6 |
| 7. TECNOLOGÍAS UTILIZADAS .....                   | 7 |
| 8. LIBRERÍAS UTILIZADAS.....                      | 8 |
| Variables de entorno utilizadas .....             | 8 |

9. ORGANIZACIÓN Y MÓDULOS DEL SISTEMA ..... 9

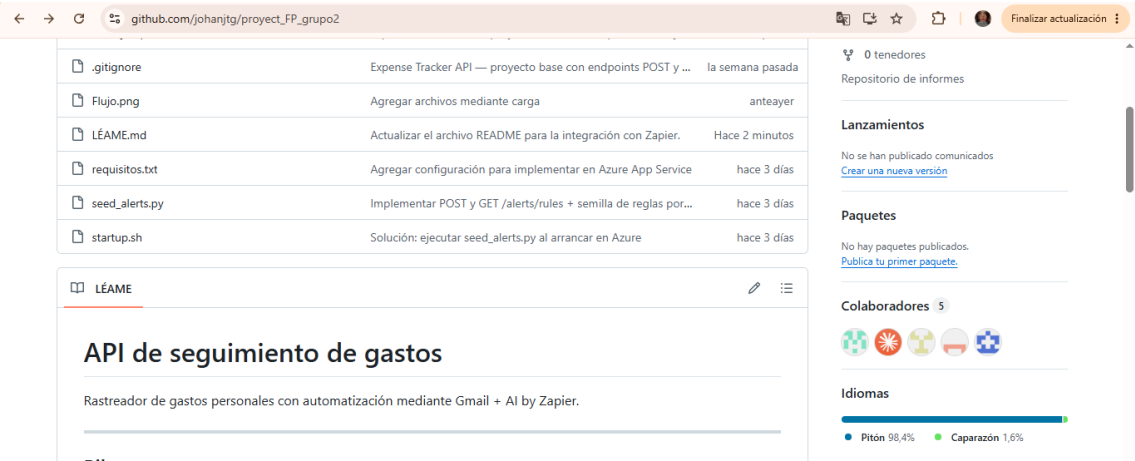
    Descripción de los módulos principales ..... 9

10. ENDPOINTS PRINCIPALES..... 11

11. INTEGRACIÓN Y AUTOMATIZACIÓN CON ZAPIER ..... 13

12. GESTIÓN DEL PROYECTO MEDIANTE GITHUB..... 14

13. EVIDENCIAS DE PRUEBAS REALIZADAS ..... 15

A screenshot of a GitHub repository page for 'Expense Tracker API'. The repository is owned by 'johanitg' and is part of a group project. The file list shows several files: '.gitignore', 'Flujo.png', 'LÉAME.md', 'requisitos.txt', 'seed\_alerts.py', and 'startup.sh'. The 'LÉAME' file is selected and its content is displayed below. The content of 'LÉAME' is titled 'API de seguimiento de gastos' and describes a personal expense tracker with automation using Gmail and Zapier. The right sidebar shows repository statistics: 0 followers, no releases, no packages, 5 collaborators, and a language distribution chart showing Python at 98.4% and Caparazón at 1.6%.

..... 16

Fuente: Elaboración propia. .... 17

14. RESULTADOS OBTENIDOS ..... 18

15. INNOVACIONES IMPLEMENTADAS ..... 19

16. LIMITACIONES DEL PROYECTO ..... 20

17. CONCLUSIONES ..... 21

18. BIBLIOGRAFÍA..... 22

19. ANEXO INDIVIDUAL ..... 23



## 1. INTRODUCCIÓN

La automatización de procesos financieros se ha convertido en una herramienta fundamental para optimizar la gestión de la información y mejorar la toma de decisiones en diferentes ámbitos económicos. En el contexto de las finanzas personales, disponer de mecanismos automatizados para registrar y analizar los gastos permite fortalecer la planificación financiera y reducir errores asociados a los procesos manuales.

Con el propósito de ofrecer una solución práctica y escalable, se desarrolló la **API de Seguimiento de Gastos Personales**, una aplicación basada en FastAPI que permite registrar gastos, clasificarlos por categorías y generar resúmenes financieros de forma estructurada.

Asimismo, el proyecto incorpora herramientas de automatización y colaboración mediante GitHub y Zapier, permitiendo organizar el desarrollo del sistema y facilitar la integración con servicios externos.

El desarrollo se realizó de manera colaborativa, utilizando GitHub como sistema de control de versiones, lo que permitió mantener un historial de cambios y evidenciar la participación de los integrantes del grupo.

## 2. CONTEXTUALIZACIÓN DEL PROBLEMA

Actualmente, muchas personas presentan dificultades para mantener un control adecuado de sus gastos debido a la ausencia de herramientas organizadas que permitan registrar y analizar la información financiera de manera eficiente.

Los registros manuales suelen generar errores, pérdida de información y limitaciones para realizar análisis posteriores. Además, el incremento de gastos asociados a servicios, suscripciones y consumo cotidiano hace necesario disponer de sistemas que permitan organizar la información y facilitar la toma de decisiones.

Ante esta problemática surge la **API de Seguimiento de Gastos Personales**, una solución orientada al registro estructurado de gastos mediante una arquitectura basada en APIs REST, permitiendo centralizar la información financiera y facilitar futuras ampliaciones del sistema.



### **3. OBJETIVO GENERAL**

Desarrollar una API para la gestión y seguimiento de gastos personales mediante el uso de FastAPI y herramientas de control de versiones, permitiendo registrar información financiera, clasificar gastos y generar resúmenes organizados para apoyar la toma de decisiones.

### **4. OBJETIVOS ESPECÍFICOS**

- Implementar una API REST para registrar gastos personales.
- Permitir la consulta de los gastos almacenados.
- Clasificar los gastos en diferentes categorías financieras.
- Generar resúmenes agrupados por categorías.
- Diseñar una estructura modular que facilite el mantenimiento del sistema.
- Organizar el desarrollo del proyecto mediante GitHub.
- Incorporar una arquitectura escalable que permita futuras integraciones con herramientas externas.
- Aplicar los conocimientos adquiridos en la asignatura Automatización de Procesos Financieros.

## **5. REQUERIMIENTOS DEL SISTEMA**

### **5.1 Requerimientos funcionales**

La **API de Seguimiento de Gastos Personales** fue desarrollada para proporcionar una solución organizada para el registro y consulta de información financiera. Los principales requerimientos funcionales definidos fueron los siguientes:

#### **RF01. Registro de gastos**

El sistema debe permitir registrar gastos mediante solicitudes HTTP POST, almacenando información relacionada con el monto, categoría, descripción y tipo de gasto.

#### **RF02. Consulta de gastos**

El sistema debe permitir consultar la información almacenada mediante solicitudes HTTP GET.

#### **RF03. Clasificación por categorías**

Los gastos deben clasificarse en categorías predefinidas para facilitar su organización y posterior análisis.

Las categorías utilizadas son:

- Food
- Subscriptions
- Transport
- Entertainment
- Health
- Other

#### **RF04. Generación de resúmenes**

El sistema debe permitir obtener resúmenes agrupados por categorías.

#### **RF05. Gestión de reglas de alerta**

La aplicación debe permitir consultar las reglas de alerta configuradas.

#### **RF06. Integración con herramientas externas**

El sistema debe facilitar futuras integraciones mediante herramientas de automatización como Zapier y Gmail.

#### **RF07. Comunicación mediante Webhooks**

La arquitectura debe permitir el intercambio de información con servicios externos.

### **5.2 Requerimientos no funcionales**

#### **RNF01. Escalabilidad**

La arquitectura debe permitir incorporar nuevas funcionalidades en futuras versiones.

**RNF02. Mantenibilidad**

El código debe estar organizado en módulos independientes para facilitar su mantenimiento.

**RNF03. Rendimiento**

La API debe responder de manera eficiente a las solicitudes realizadas por los usuarios.

**RNF04. Portabilidad**

La aplicación debe poder ejecutarse tanto en entornos locales como en servicios en la nube.

**RNF05. Colaboración**

El desarrollo debe permitir el trabajo colaborativo mediante herramientas de control de versiones.

## 6. ARQUITECTURA GENERAL DEL SISTEMA

La **API de Seguimiento de Gastos Personales** fue desarrollada utilizando una arquitectura basada en servicios REST, permitiendo separar las diferentes responsabilidades del sistema y facilitar su mantenimiento.

La arquitectura general se encuentra compuesta por los siguientes elementos:

### Usuario

Representa al cliente que realiza solicitudes al sistema.

### FastAPI

Constituye el núcleo principal de la aplicación y gestiona la lógica del negocio.

### Base de datos

Permite almacenar la información correspondiente a los gastos y reglas configuradas.

### Zapier

Permite ampliar las capacidades del sistema mediante flujos automatizados.

### Azure App Service

Se incorporó la configuración necesaria para permitir futuros despliegues en la nube.

### Flujo general del sistema

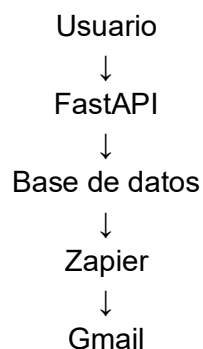
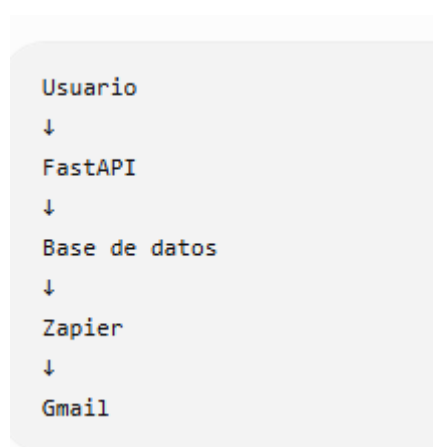


Figura 1. Arquitectura general del sistema.



Fuente: Elaboración propia.

## 7. TECNOLOGÍAS UTILIZADAS

Durante el desarrollo del proyecto se emplearon diferentes tecnologías para la construcción de la API y la gestión colaborativa del código.

| Tecnología        | Descripción                                   |
|-------------------|-----------------------------------------------|
| Python            | Lenguaje principal del proyecto               |
| FastAPI           | Framework para el desarrollo de APIs REST     |
| SQLite            | Base de datos utilizada durante el desarrollo |
| GitHub            | Control de versiones y colaboración           |
| Gmail             | Fuente de correos electrónicos                |
| Zapier            | Automatización e integración                  |
| Azure App Service | Configuración para futuros despliegues        |
| Webhooks          | Comunicación con herramientas externas        |

## 8. LIBRERÍAS UTILIZADAS

La implementación de la **API de Seguimiento de Gastos Personales** requirió el uso de diferentes librerías que facilitaron el desarrollo, validación y organización del sistema.

### **FastAPI**

Framework utilizado para la construcción de la API REST.

### **SQLAlchemy**

Librería ORM empleada para la gestión de la base de datos y la interacción con los modelos.

### **Pydantic**

Permite validar y estructurar los datos recibidos por la API.

### **Uvicorn**

Servidor ASGI utilizado para ejecutar la aplicación.

### **Python-dotenv**

Facilita la gestión de las variables de entorno del proyecto.

### **Requests**

Permite realizar solicitudes HTTP hacia servicios externos.

### **SQLite3**

Base de datos ligera utilizada durante las pruebas y el desarrollo local.

### **Variables de entorno utilizadas**

| Variable           | Descripción                                                        |
|--------------------|--------------------------------------------------------------------|
| DATABASE_URL       | Cadena de conexión de la base de datos                             |
| ZAPIER_WEBHOOK_URL | Webhook utilizado por Zapier para enviar la información procesada. |

## 9. ORGANIZACIÓN Y MÓDULOS DEL SISTEMA

Con el objetivo de mantener una estructura organizada y facilitar el mantenimiento del proyecto, la aplicación fue desarrollada siguiendo una arquitectura modular.

La estructura principal del repositorio es la siguiente:

project\_FP\_grupo2/

```
|
|— app/
|   |— main.py
|   |— models.py
|   |— schemas.py
|   |— database.py
|   |— alerts.py
|   |— routes/
|
|— .github/
|
|— requirements.txt
|— README.md
|— seed_alerts.py
|— startup.sh
|— Flujo.png
```

### Descripción de los módulos principales

#### **main.py**

Archivo principal encargado de iniciar la aplicación FastAPI y registrar las rutas del sistema.

#### **models.py**

Contiene los modelos de datos utilizados por la base de datos.

#### **schemas.py**

Define los esquemas de validación implementados mediante Pydantic.

#### **database.py**

Gestiona la conexión con la base de datos.

## alerts.py

Implementa la lógica asociada a las reglas de alerta.

## routes/

Agrupar los diferentes endpoints disponibles.

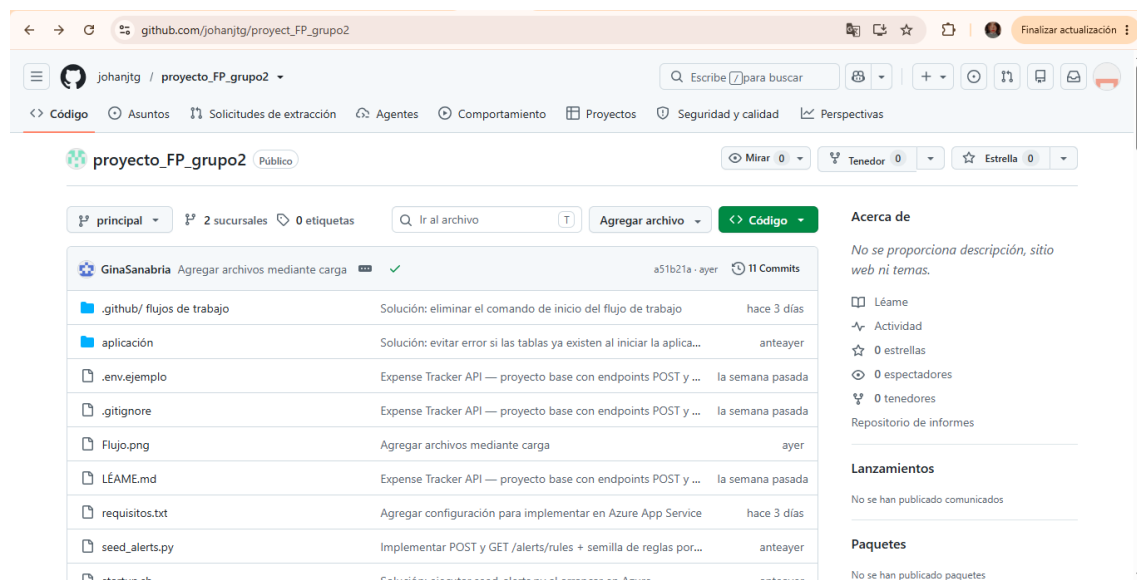
## seed\_alerts.py

Permite cargar reglas iniciales para las categorías de gasto.

## .github/

Contiene los archivos relacionados con el control de versiones y la configuración del proyecto.

Figura 2. Estructura general del repositorio del proyecto.



Fuente: Elaboración propia.



## 10. ENDPOINTS PRINCIPALES

La API fue desarrollada bajo una arquitectura REST, permitiendo la interacción con el sistema mediante diferentes solicitudes HTTP.

### Endpoint 1: Registro de gastos

#### Método

POST /expenses

#### Descripción

Permite registrar un nuevo gasto dentro del sistema.

#### Ejemplo de solicitud

```
{  
  "amount": 15.99,  
  "category": "Food",  
  "description": "Burger King",  
  "is_subscription": false  
}
```

### Endpoint 2: Consulta de gastos

#### Método

GET /expenses

#### Descripción

Permite consultar la lista de gastos almacenados.

### Endpoint 3: Resumen por categorías

#### Método

GET /expenses/summary

#### Descripción

Genera un resumen agrupado por categorías financieras.

### Endpoint 4: Reglas de alerta

#### Método

GET /alerts/rules

#### Descripción

Permite consultar las reglas configuradas para cada categoría.

## **Endpoint 5: Webhooks**

### **Descripción**

Permiten enviar información hacia servicios externos para facilitar procesos de automatización.

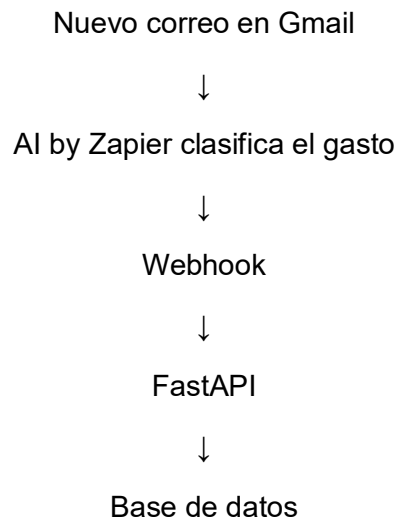
## 11. INTEGRACIÓN Y AUTOMATIZACIÓN CON ZAPIER

Uno de los principales objetivos del proyecto fue incorporar herramientas de automatización para ampliar las capacidades del sistema.

### Integración con Zapier

El proyecto incorpora una integración mediante Zapier con el objetivo de automatizar procesos y facilitar la comunicación con servicios externos.

El flujo general implementado es:



La utilización de Zapier permite extender las capacidades del sistema y simplificar la automatización de determinadas tareas.

### Innovación incorporada

La combinación de FastAPI, GitHub y Zapier permitió desarrollar una solución flexible y escalable para la gestión de gastos personales.

## 12. GESTIÓN DEL PROYECTO MEDIANTE GITHUB

El desarrollo de la **API de Seguimiento de Gastos Personales** se realizó de forma colaborativa utilizando GitHub como plataforma de control de versiones y repositorio central del proyecto.

La utilización de GitHub permitió:

- Mantener un historial cronológico del desarrollo.
- Registrar cambios mediante commits.
- Facilitar el trabajo colaborativo entre los integrantes.
- Centralizar el código fuente y la documentación.
- Mantener una estructura organizada del repositorio.

### Repositorio del proyecto:

[https://github.com/johanjtg/proyect\\_FP\\_grupo2](https://github.com/johanjtg/proyect_FP_grupo2)

La estructura del repositorio incluye:

- Código fuente del sistema.
- Archivos de configuración.
- Dependencias necesarias para la ejecución.
- Documentación del proyecto.
- Evidencias del desarrollo.

Asimismo, GitHub permitió evidenciar la evolución progresiva del sistema mediante múltiples commits realizados durante la implementación.

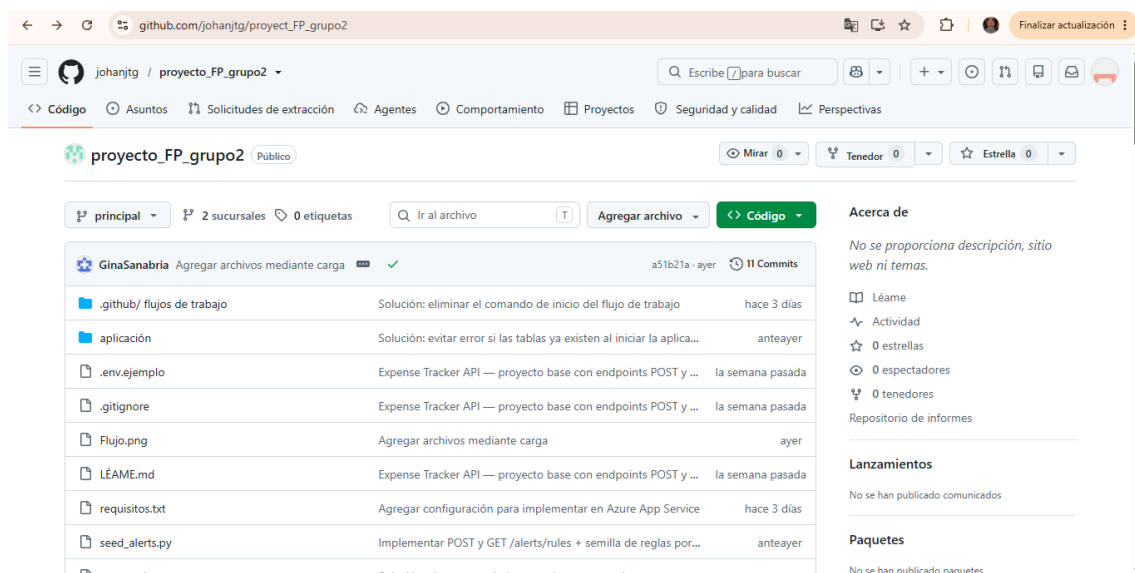
### 13. EVIDENCIAS DE PRUEBAS REALIZADAS

Con el objetivo de verificar la organización y funcionamiento del sistema, se recopilaron evidencias asociadas al desarrollo del proyecto.

#### Evidencia 1. Repositorio principal del proyecto

Se verificó la estructura general del repositorio y los archivos principales del sistema.

Figura 3. Repositorio principal en GitHub.

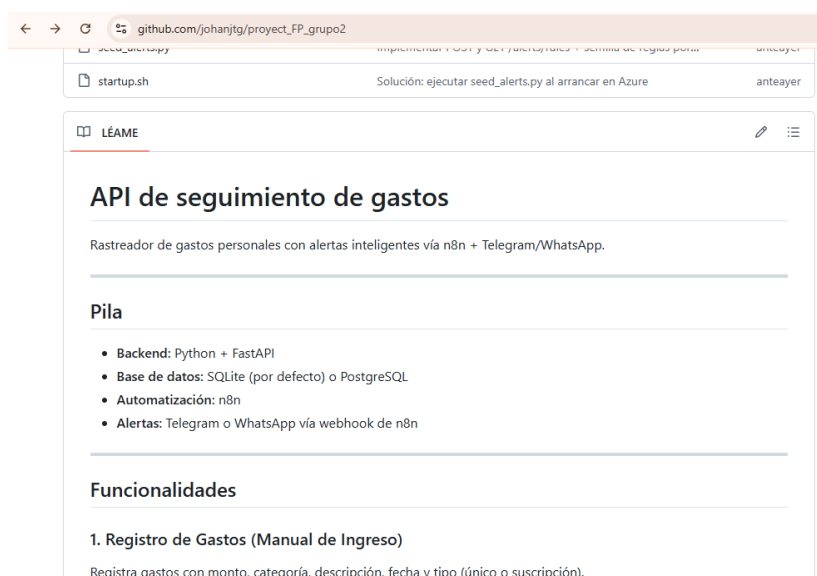


Fuente: Elaboración propia.

#### Evidencia 2. README del proyecto

Se revisó la documentación principal y las funcionalidades descritas en el repositorio.

Figura 4. Documentación del proyecto.

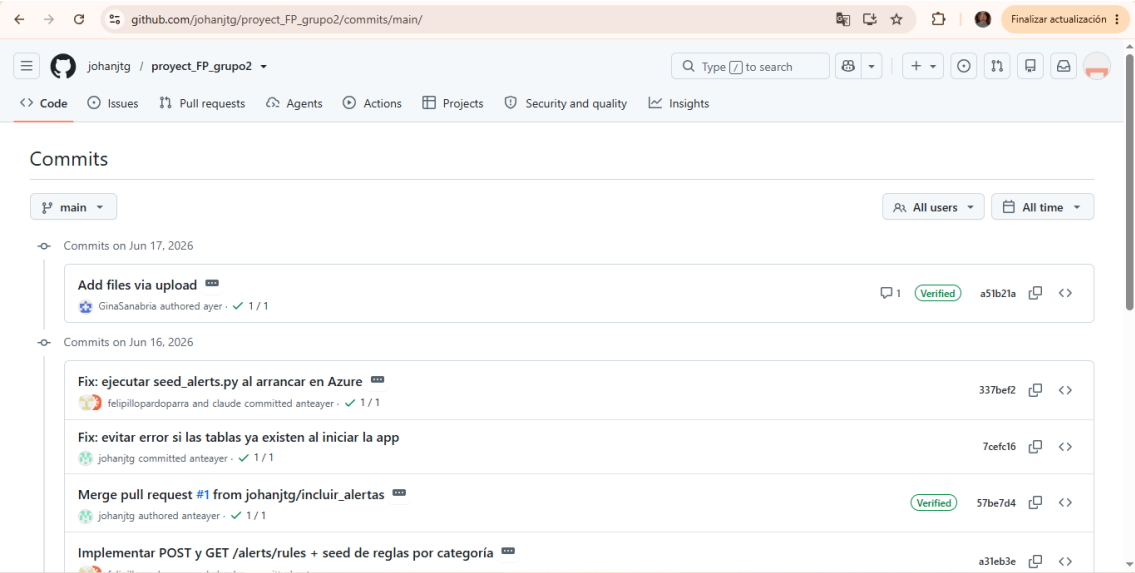


Fuente: Elaboración propia.

Evidencia 3. Historial de commits

Se comprobó el historial de cambios realizados por los integrantes del equipo.

Figura 5. Historial de commits del proyecto.

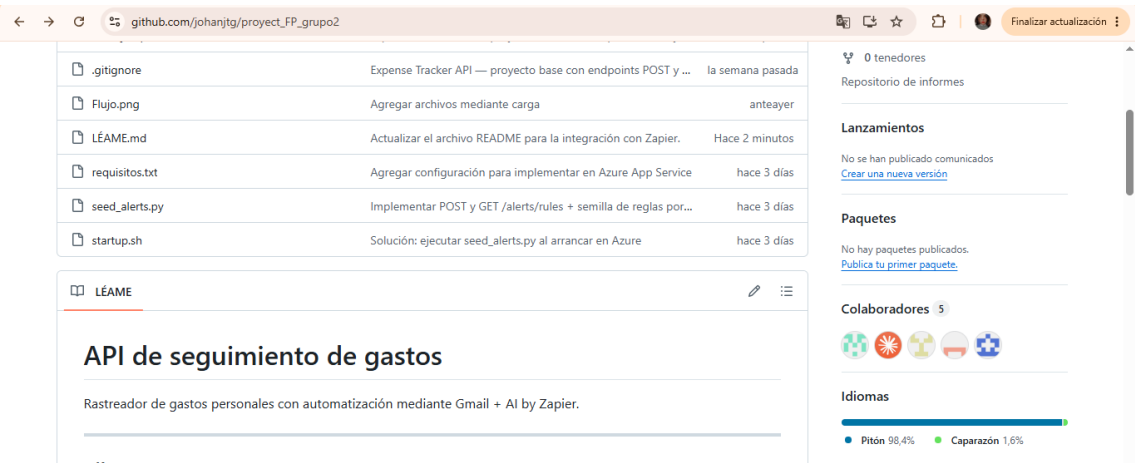


Fuente: Elaboración propia.

Evidencia 4. Colaboración entre integrantes

Se verificó la participación de los colaboradores registrados en GitHub.

Figura 6. Colaboradores del repositorio.



Fuente: Elaboración propia.

Evidencia 5. Variables de entorno y organización del sistema

Se revisó la estructura interna del proyecto y las configuraciones utilizadas.

Figura 7. Organización del sistema y variables de entorno.

← →

github.com/johanjtg/proyct\_FP\_grupo2

LEÁME

API de seguimiento de gastos

Rastreador de gastos personales con alertas inteligentes vía n8n + Telegram/WhatsApp.

Pila

- Backend: Python + FastAPI
- Base de datos: SQLite (por defecto) o PostgreSQL
- Automatización: n8n
- Alertas: Telegram o WhatsApp vía webhook de n8n

Funcionalidades

1. Registro de Gastos (Manual de Ingreso)

Registra gastos con monto, categoría, descripción, fecha y tipo (único o suscripción).

Categorías: Food, Subscriptions, Transport, Entertainment, Health, Other

← →

github.com/johanjtg/proyct\_FP\_grupo2

LEÁME

Instalación

```
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload
```

Variables de Entorno

| Variable        | Descripción                         | Valor por defecto                           |
|-----------------|-------------------------------------|---------------------------------------------|
| DATABASE_URL    | Cadena de conexión SQLAlchemy       | sqlite:///./expenses.db                     |
| N8N_WEBHOOK_URL | URL del webhook de n8n para alertas | http://localhost:5678/webhook/expense-alert |

Fuente: Elaboración propia.

Figura 8. Flujo automatizado implementado mediante Zapier para el procesamiento de información Financiera.

← →

github.com/johanjtg/proyct\_FP\_grupo2/blob/main/Flujo.png

Finalizar actualización

Archivos

principal

Ir al archivo

- .github/ flujos de trabajo
- aplicación
  - .env.ejemplo
  - .gitignore
  - Flujo.png
  - LEÁME.md
  - requisitos.txt
  - seed\_alerts.py
  - startup.sh

68,6 KB

Version: v3 Run ID: 01a87f8c-cf78-a348-7f18-b5f30a056ad4

Successful: jun 17, 2026 18:14:57

✓ Gmail 2 min

1. New Email

Found 1 new Email in Gmail.

✓ AI by Zapier

2. Analizar y clasificar gasto del correo

Sent 1 new Results to AI by Zapier.

✓ Webhooks by Zapier

Fuente: Elaboración propia.

## **14. RESULTADOS OBTENIDOS**

Como resultado del desarrollo del proyecto se logró implementar una solución organizada para la gestión de gastos personales.

Los principales resultados obtenidos fueron:

- Implementación de una API basada en FastAPI.
- Registro y consulta de gastos mediante endpoints REST.
- Clasificación de gastos por categorías.
- Generación de resúmenes financieros.
- Incorporación de reglas de alerta.
- Integración con herramientas de automatización.
- Organización del desarrollo mediante GitHub.
- Elaboración de documentación técnica del sistema.
- Implementación de un flujo automatizado mediante Gmail y Zapier.

El proyecto desarrollado proporciona una base sólida para futuras ampliaciones y nuevas funcionalidades.



## **15. INNOVACIONES IMPLEMENTADAS**

Además de las funcionalidades básicas, se incorporaron elementos que incrementan la escalabilidad del sistema.

Las principales innovaciones fueron:

### **Arquitectura modular**

La organización del código facilita el mantenimiento y la reutilización de componentes.

### **Uso de FastAPI**

Permite construir APIs modernas y eficientes.

### **Integración con Zapier**

Facilita la automatización de procesos y futuras integraciones.

### **Configuración para Azure App Service**

Se incorporaron los archivos necesarios para permitir futuros despliegues en la nube.

### **Uso de GitHub**

Facilitó el trabajo colaborativo y el control de versiones.

### **Integración con Inteligencia Artificial**

La utilización de AI by Zapier permitió clasificar automáticamente la información proveniente de los correos electrónicos antes de enviarla a la API.

## **16. LIMITACIONES DEL PROYECTO**

A pesar de los resultados obtenidos, se identificaron algunas limitaciones:

- El sistema se encuentra orientado a gastos personales y no empresariales.
- Las reglas de alerta implementadas son básicas y pueden ampliarse en futuras versiones.
- La base de datos utilizada durante el desarrollo corresponde a un entorno de pruebas.
- Las integraciones externas pueden ampliarse con nuevos servicios.
- El sistema puede evolucionar hacia arquitecturas más complejas en futuras implementaciones.

## **17. CONCLUSIONES**

### **Conclusión 1.**

FastAPI permitió desarrollar una API organizada y fácilmente escalable.

### **Conclusión 2.**

La arquitectura modular facilita el mantenimiento y futuras ampliaciones.

### **Conclusión 3.**

GitHub permitió mantener un control adecuado del desarrollo y evidenciar el trabajo colaborativo.

### **Conclusión 4.**

La integración con herramientas de automatización incrementa la flexibilidad del sistema.

### **Conclusión 5.**

El proyecto permitió aplicar de forma práctica los conocimientos adquiridos en la asignatura Automatización de Procesos Financieros.

### **Conclusión 6.**

El trabajo colaborativo permitió integrar diferentes conocimientos técnicos y obtener una solución funcional orientada a la automatización de procesos financieros.

Para el desarrollo del proyecto se consultaron diversas fuentes oficiales relacionadas con FastAPI, Python, GitHub y las herramientas de automatización utilizadas, las cuales permitieron sustentar técnicamente la implementación realizada.

## 18. BIBLIOGRAFÍA

FastAPI. (2026). \*FastAPI Documentation\*. <https://fastapi.tiangolo.com/>

GitHub. (2026). *GitHub Documentation*. <https://docs.github.com/>

Microsoft. (2026). *Azure Documentation*. <https://learn.microsoft.com/azure/>

Pydantic. (2026). *Pydantic Documentation*. <https://docs.pydantic.dev/>

Python Software Foundation. (2026). *Python Documentation*. <https://www.python.org/>

SQLAlchemy. (2026). *SQLAlchemy Documentation*. <https://www.sqlalchemy.org/>

Zapier. (2026). *Zapier Documentation*. <https://zapier.com/>

## **19. ANEXO INDIVIDUAL**

### **Integrante**

**Cindy Marilú Pablo Espinoza**

### **Participación dentro del proyecto**

Mi participación estuvo orientada principalmente a la elaboración y organización de la documentación técnica y funcional del proyecto.

Las actividades realizadas incluyeron:

- Elaboración del informe final.
- Organización de las evidencias.
- Revisión de la arquitectura del sistema.
- Descripción de módulos y endpoints.
- Documentación de las integraciones con Gmail y Zapier.
- Elaboración de resultados y conclusiones.

### **Reflexión personal**

El desarrollo de este proyecto permitió fortalecer mis conocimientos sobre APIs REST, automatización de procesos financieros y trabajo colaborativo mediante GitHub.

Asimismo, comprendí la importancia de la documentación técnica como parte fundamental del ciclo de desarrollo de software y la relevancia de las herramientas de automatización para optimizar procesos.

### **Integrante**

**Gina Julieth Sanabria Maldonado**

### **Participación dentro del proyecto**

Mi aporte al proyecto consistió en desarrollar un flujo de automatización que integra tres herramientas: Gmail, inteligencia artificial (AI by Zapier) y la API de Seguimiento de Gastos Personales.

El proceso funciona así:

- El sistema detecta automáticamente cuando llega un nuevo correo en Gmail relacionado con un gasto.
- Ese correo es procesado por un modelo de IA que analiza y clasifica el tipo de gasto.
- Finalmente, la información clasificada se envía de forma automática a la API de SmartBudget para su registro.

### **Reflexión personal**

La implementación del flujo automatizado mediante Gmail y AI by Zapier me permitió fortalecer mis conocimientos sobre automatización de procesos y aplicaciones de

inteligencia artificial en sistemas financieros. Asimismo, comprendí la importancia de integrar diferentes servicios para optimizar tareas repetitivas.

### **Integrante**

**Johan Javier Trujillo Gutiérrez**

### **Participación dentro del proyecto**

Mi aporte al proyecto es la definición de arquitectura:

- Desarrollo servicio de summary y versión base del producto

### **Reflexión personal**

El desarrollo de la arquitectura del sistema y de los servicios principales permitió consolidar conocimientos sobre APIs REST y diseño modular. Además, el trabajo colaborativo facilitó la integración de las diferentes contribuciones realizadas por el equipo.